

Messaging Modernization

Migrating from JMS Brokers (TIBCO IBM ,MQ etc.) & Solace to NATS

Bruno Baloi · Solutions Strategy Lead

October 8,2026 · Synadia



WHO'S TALKING

Bruno Baloi

LEAD SOLUTIONS STRATEGY

RELEVANT EXPERIENCE

+25 years in Integration

Worked with most Messaging providers
commercial and OSS.

(IBM MQ, TIBCO, Solace, Active MQ, Kafka etc.)

KEEP IN MIND

- Migrations aren't easy, but they are often ***necessary***.
- ***Market demands*** and new use cases often drive the need for change
- It is important to understand the product differences, the challenges and know the migration roadmap.

What we'll do today

01

A concept-mapping cheat sheet

Topics, queues, guaranteed messaging — mapped to NATS, side by side.

02

A migration checklist

feature parity, clustering, scaling, clients

03

Hands-on exercise

On some of the mapped concepts

01

WHY

What's actually driving this decision — and whether it's the same thing driving yours.

Companies demand more from their (RT) messaging

The business landscape is changing. Many external factors are driving the need for technology modernization.

OPPORTUNITY

	Customer experience	notifications	cross-sell	upsell
	Situational awareness	predictive maintenance	fraud detection	
	Real-time visibility	logistics	track and trace	
	Real-time decision making	event processing	analytics	
	Agentic AI automation	agent-to-agent (A2A) messaging		

Growth

- exp
- ne
- op

Messa

- Re
- Dri

- Growth=**
- expand to new markets
 - new product/services
 - operational efficiencies

- Messaging =**
- Real Time Interaction
 - Drives engagement

OBLIGATION

	Regulatory compliance	GDPR	CCPA	HIPAA
	Cybersecurity	threat exposure	data breach risk	

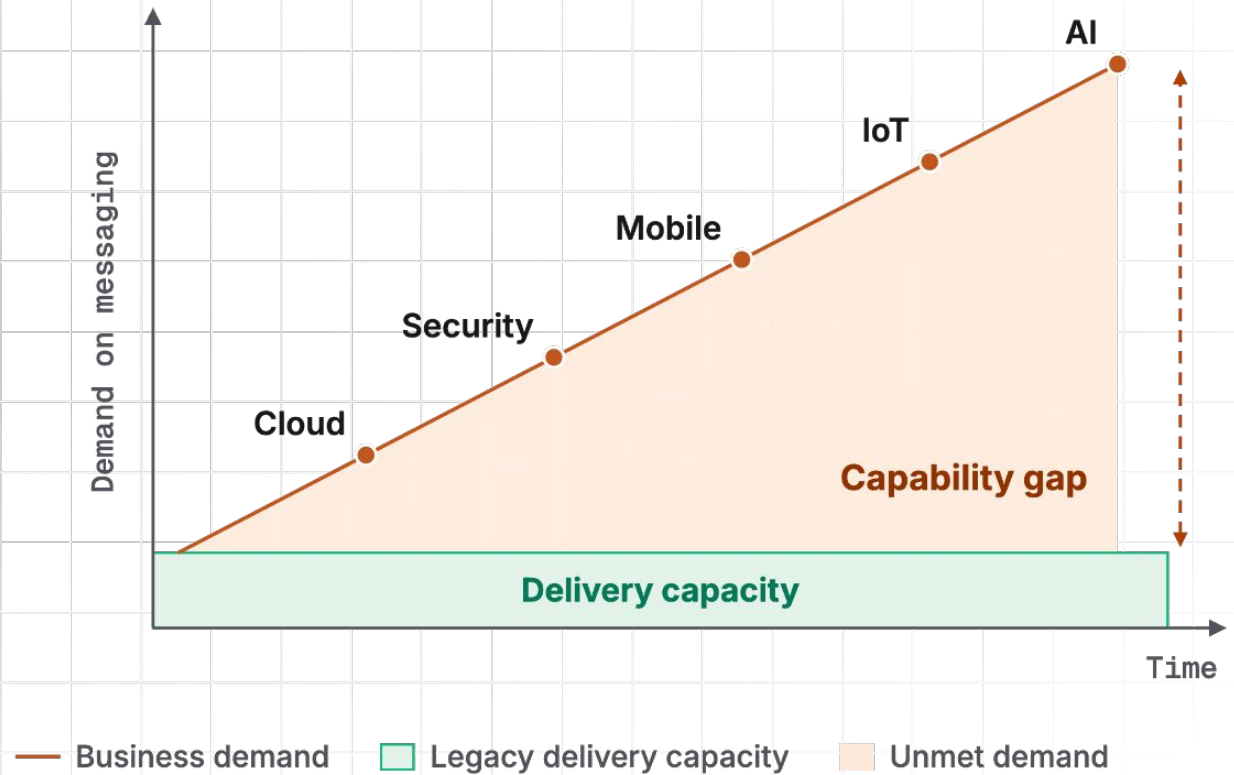
(Legacy) Messaging Systems

Challenged by these demands

WHERE LEGACY MESSAGING FALLS SHORT

- 01 Enterprise-bound**
Not designed to scale out to the cloud
- 02 Fragmented connectivity**
Cloud and edge each need another messaging system
- 03 Sized for yesterday**
Built for enterprise volumes and latencies
- 04 Brittle security**
Perimeter models that break outside the data center
- 05 No streaming**
Messaging and streaming live in separate stacks

DEMAND OUTPACES DELIVERY



Migration drivers

Nobody migrates for fun! Three things put this on a roadmap. Usually two at once.



Commercial pressure

- *Renewal terms that scale with volume rather than value.*
- *Rising infra costs*
- *Poor customer experience*
- Hard to deliver new services on budget



Architectural drift

- *Multi Cloud/enterprise fabric requirements*
- *Edge requirements*
- Protocol obsolescence
- Performance & scalability



Operational drag

- *Specialist skills that are hard to hire.*
- *Consolidation:*
 - Multiple messaging platforms
 - Solutions requires multiple products
- Appliance dependencies that dictate where things can run.

02

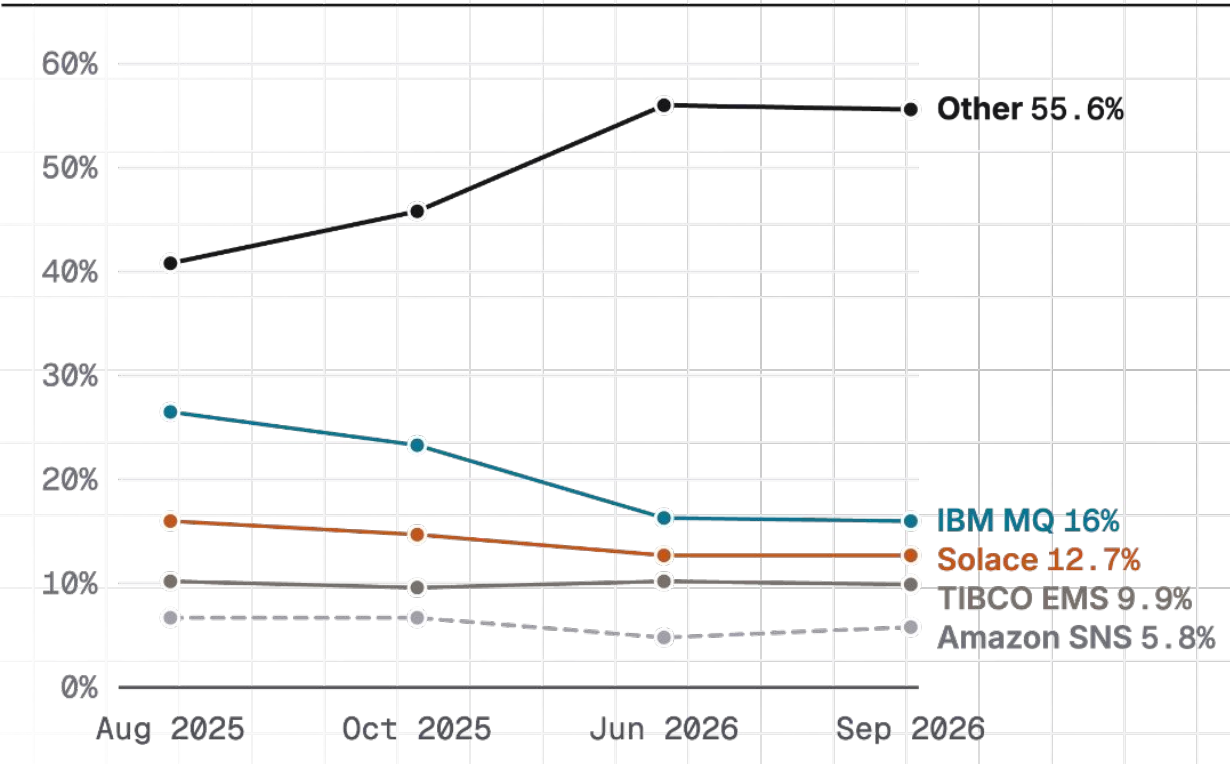
What

Legacy products warranting migration

State of the (messaging) world

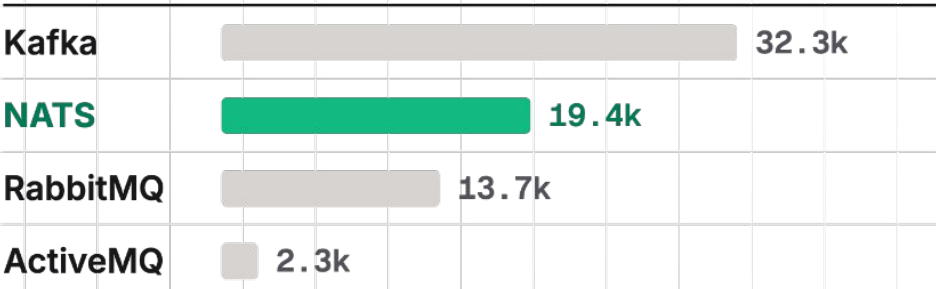
- 1. RabbitMQ/IBM MQ/JMS/RV
- 2. Kafka
- 3. Solace

MINDSHARE · MESSAGE-ORIENTED MIDDLEWARE



Source: PeerSpot · dashed = approximate

GITHUB STARS · CORE REPO



WHAT "OTHER" INCLUDES

TIBCO Rendezvous	Aurea CX Messenger
Amazon EventBridge	RTI Connexx DDS
Amazon SNS	Tervela Data Fabric
TIBCO FTL	Software AG Universal Messaging
Oracle BAM	Azure Event Grid
Confluent	RedPanda

JMS Brokers

(TIBCO EMS, AMQ, IBM MQ, etc.)



JMS Spec is 25+ years old and dates back to the J2EE world

No protocol interop across vendors

JMS BROKERS · TWO CATEGORIES

Native JMS

TIBCO EMS ActiveMQ
Oracle WebLogic IBM WebSphere

JMS-compatible

IBM MQ Solace PubSub+

KEY FEATURES

- Designed for the enterprise
- Active-passive clustering
- Moderate performance
- Large JVM footprint
- Point-to-point queues and pub/sub topics
- Guaranteed delivery

KEY MIGRATION REASONS

- 01 Rising operating cost**
Operations and maintenance costs keep climbing
- 02 Lock-in**
Language and protocol lock-in; brokers don't interoperate
- 03 Selectors don't scale**
Message selectors become the bottleneck as load grows
- 04 Inverted design**
Long-lived connections vs. elastic, container-native workloads
- 05 Broker-centric scaling**
Scale is tied to the broker, not partition-native
- 06 Poor cloud observability**
A weak fit for cloud-native monitoring

Solace

Originally a MOM queue server.



Solace and IBM initially both began as MOM (queue servers) with IBM layering JMS on top, and Solace starting out as an XML Router, and laying over a subject based addressing model (borrowed from TIBCO Rendezvous)

SOLACE · TWO OFFERINGS

Appliance	3530	3560
Software broker	PubSub+	OSS Docker image

KEY FEATURES

- Subject-based addressing
- Designed for high performance
- Event mesh with DMR
- Queues, topics, request-reply and replay
- Guaranteed delivery
- **Partitioned queues**
- Active-passive clustering
- Large footprint (container-only)

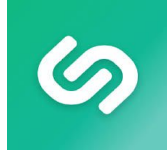
KEY MIGRATION REASONS

- 01 **High cost**
Both the appliances and PubSub+ are expensive
- 02 **Uncertain appliance future**
- 03 **Vendor lock-in**
- 04 **Cost of streaming requirements**
- 05 **VPNs and layered security**
A complex, multi-layer security model
- 06 **Complex guaranteed delivery**
Topics layered over queues
- 07 **Limited mesh clustering**
Maximum of 15 nodes per cluster
- 08 **Unsupported edge use cases**

Why pick



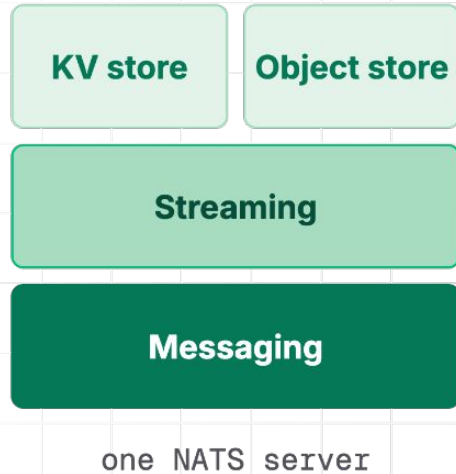
+



?

Modern, ultra-scalable, super-performant, Messaging Fabric, designed to extend from edge to the cloud to the Enterprise.

NATS · ALL IN ONE



All in one

Messaging, streaming, KV and object in one server

Size

One small binary, from device to cloud

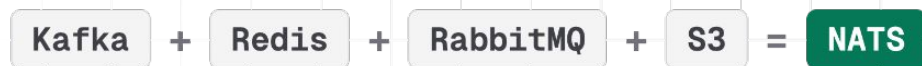
Topology

Clusters, superclusters and leaf nodes as one event fabric

Decentralized security

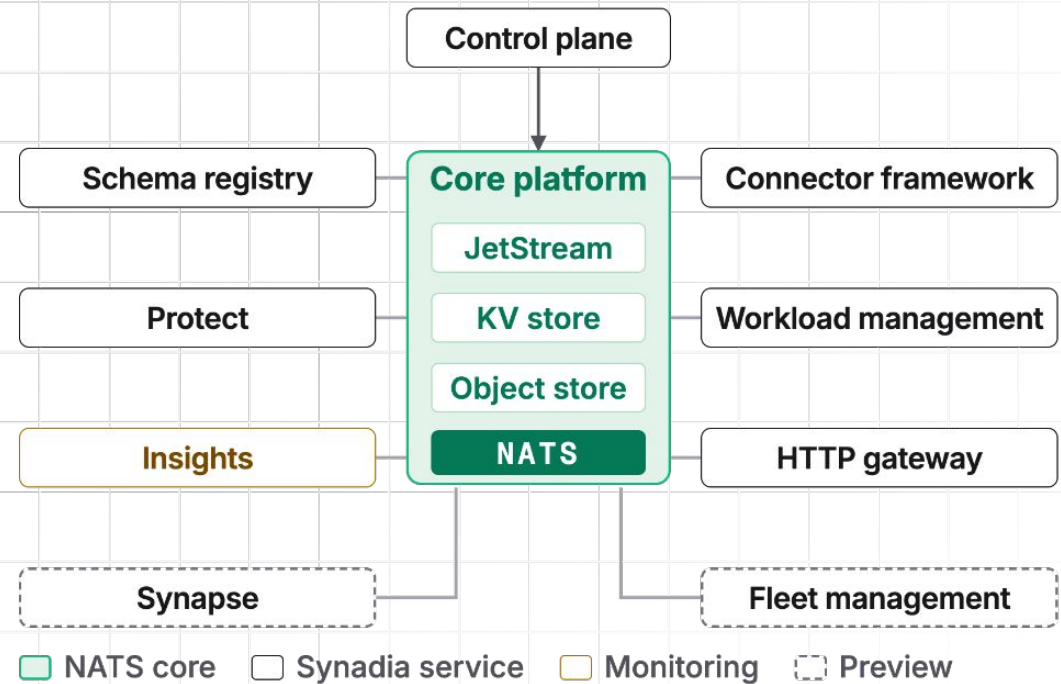
Accounts and JWTs, delegated rather than centralized

REPLACES A STACK OF POINT TOOLS



SYNADIA PLATFORM · BUILT ON NATS

- Management plane
- Enhanced monitoring
- Enhanced security
- Enhanced connectivity

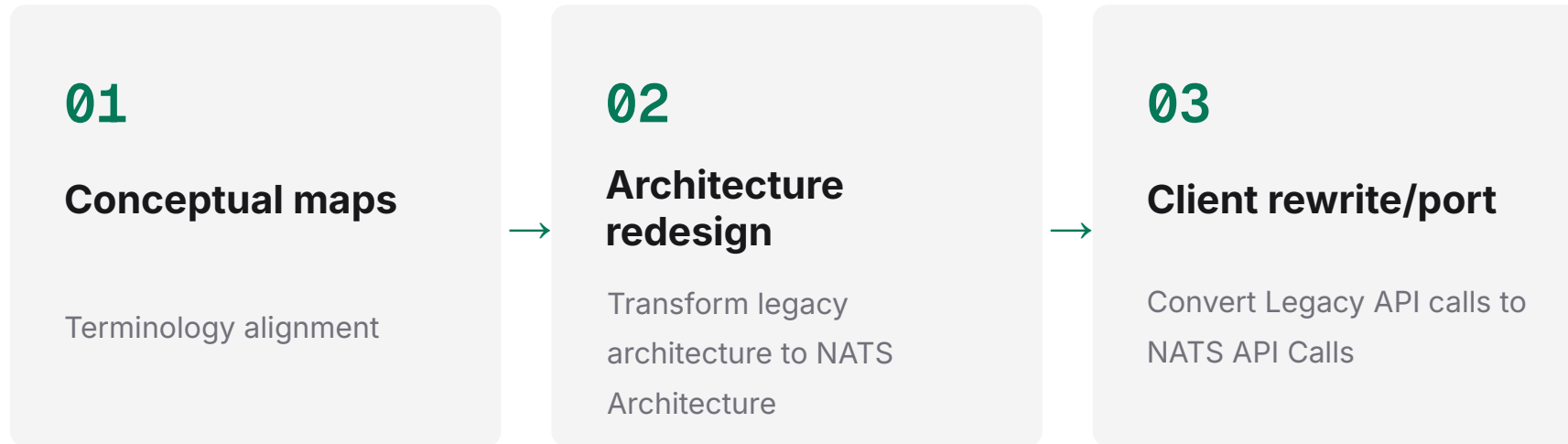


02

HOW

The mapping, the path, and the parts that bite.

Migration Process



Every step needs a verification criterion. "Then the cut over" is not a step.

What maps to what (JMS)

Most of the migration is vocabulary. This table is the cheat sheet you're taking home.

JMS CONCEPT		NATS EQUIVALENT	WHERE IT ISN'T 1:1
Topic	→	Subject	Wildcard rules differ
Queue	→	Queue group	The consumers are the queue; no separate object to manage
Durable subscriber	→	Durable consumer	Ack policy and replay position become explicit choices
Message selector	→	Subject-based filtering	Every subject token can be a filter
Guaranteed delivery	→	JetStream	Durability belongs to the stream, not a flag on the message
Temporary queue (req-reply)	→	_INBOX subject (req-reply)	A lightweight subject vs. a server-managed destination
Dead letter queue	→	Max deliver + advisory	Dead-lettering is a consumer policy, not a destination
ConnectionFactory	→	Connection	A lightweight object vs. a server-administered object

What maps to what (Solace)

Most of the migration is vocabulary. This table is the cheat sheet you're taking home.

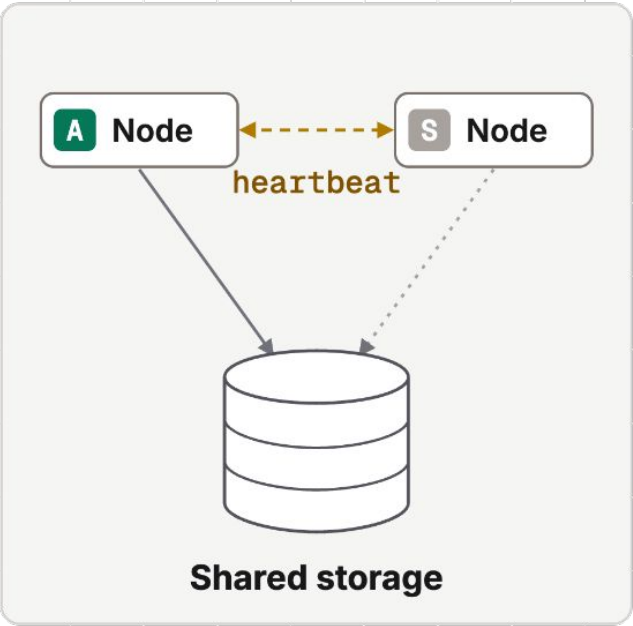
SOLACE CONCEPT		NATS EQUIVALENT	WHERE IT ISN'T 1:1
Topic	→	Subject	Wildcard rules differ — see the gotchas
Queue	→	Queue group / stream	The consumers are the queue; no separate object to manage
Subject-based filtering	→	Subject-based filtering	1:1
Message VPN	→	Account	One security structure vs. a complex layered model
Client + ACL profile	→	User (signed JWT)	Decentralized vs. broker-bound security
Guaranteed delivery	→	JetStream	Durability belongs to the stream, not a flag on the message
Partitioned queues	→	Partitioned consumer groups	Client-coordinated over a stream vs. split broker-side storage
DMR bridge	→	Leaf node / gateway	Multiple cross-cluster links vs. a single point of failure
Event mesh	→	Supercluster + leaf nodes	Edge + cloud + on-prem vs. cloud + on-prem

Architectural Map - HA/FT

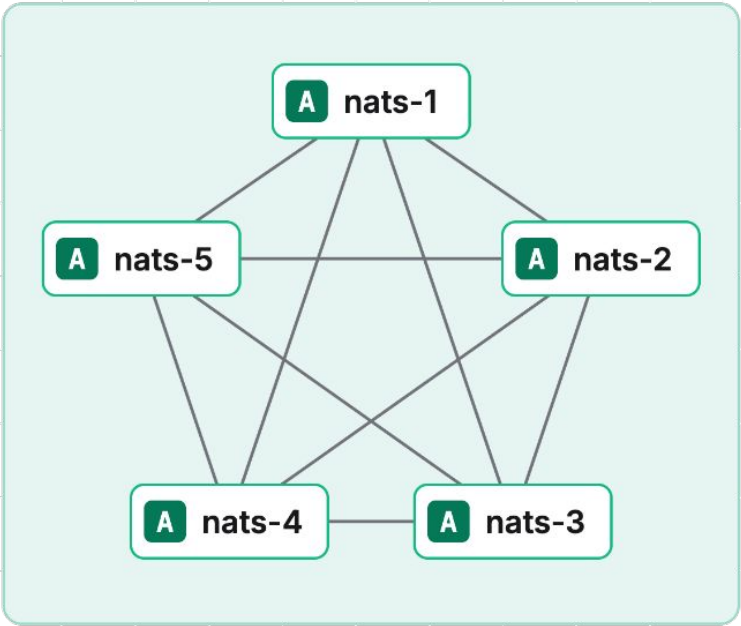
SCALING MODEL

- 01 **JMS scales by pairs**
Scaling out means load-balancing multiple fault-tolerant pairs
- 02 **NATS is active-active**
Every node serves traffic. To scale, add nodes
- 03 **JetStream is Raft-based**
Replicated streams reach consensus without shared storage

HA PAIR · EMS / MQ / SOLACE



NATS · FULL-MESH CLUSTER



— Server route - - - - - Heartbeat Idle until failover A Active S Standby

Architectural Map - Solace DMR

INTER-CLUSTER LINKS

One gateway per cluster

Solace routes each DMR cluster through a single gateway node, a chokepoint

No 15-node ceiling

Solace caps a DMR cluster at 15 nodes; NATS has no such limit

Selective replication

Solace propagates guaranteed messages across DMR. NATS mirrors or sources only the JetStream streams you choose

No single point of failure

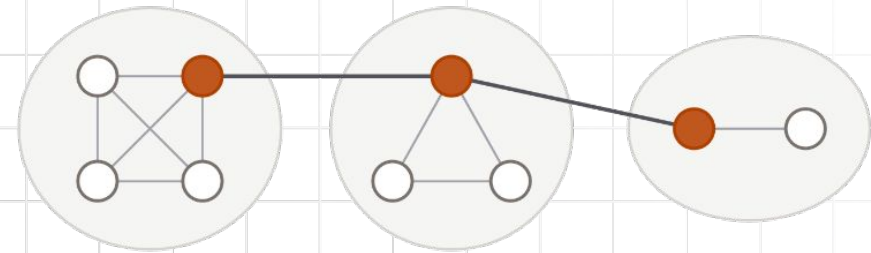
If one gateway connection drops, the others keep the inter-cluster link alive

NATS GATEWAY CONFIG

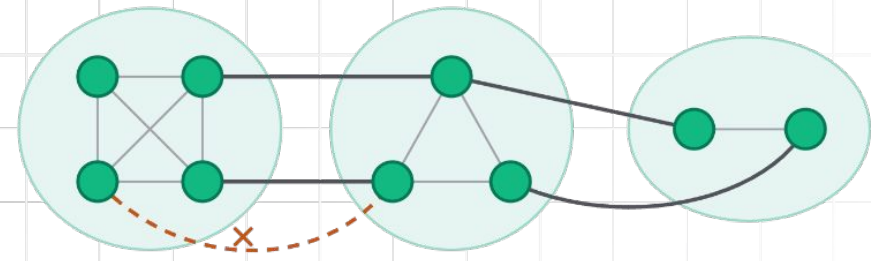
```
gateway {
  name: east
  port: 7222
  gateways: [
    { name: "west", urls: [
      "nats://n1-west:7322",
      "nats://n2-west:7323",
      "nats://n3-west:7324"
    ]}
  ]
}
```

● Gateway node - - - - - Dropped link

SOLACE DMR · ONE GATEWAY EACH



NATS · EVERY NODE A GATEWAY



Client porting

JMS AND SOLACE CLIENTS

JMS clients

TIBCO · IBM · Solace

Java

Solace clients

C .NET (C#) Go iOS Java
JavaScript JMS Node.js Python

Solace protocols

MQTT WebSockets REST AMQP
JMS

CONCLUSION

JMS and AMQP clients will generally have to be ported or rewritten

NATS CLIENTS

Tier 1

Synadia supported

Go JavaScript/TypeScript Python Java
Rust C#/.NET C

Tier 2

Zig Swift Ruby Elixir

Community

Crystal Dart Kotlin PHP Pascal
Perl Lua Nim Scala Tcl

NATS PROTOCOL SUPPORT

Protocols

MQTT WebSocket REST



Selectors 1

JMS SELECTORS

- + Rich, flexible SQL-92 syntax
 - + Decouples publishers from filtering logic
 - + Logic adjustable at runtime
-
- Doesn't scale: filtering cost grows with message volume
 - Error-prone
 - Implementation varies by vendor

NATS SUBJECT FILTERING

- + Near-zero filtering cost
 - + Filtering logic is visible in the subject
 - + Flexible filtering without per-server config
 - + Scales linearly
-
- Subject taxonomy must be designed up front
 - Comparisons must be discretized
 - Couples publishers to filter logic

+ Pro - Con

Selectors give you **runtime flexibility at the cost of broker CPU**. Subject filtering gives you **near-zero broker cost at the cost of upfront design**.

Selectors 2

THE MESSAGE

```
{
  "orderid": "12345",
  "item": "tag789",
  "price": 320,
  "store": "abc333",
  "city": "SF",
  "state": "CA"
}
```

VALUE BANDS

low_value	100-200
medium_value	200-300
high_value	300+ ← 320

SUBJECT TAXONOMY

Pattern	orders.<state>.<city>.<store>.<value>
This message	orders.ca.sf.abc333.high_value

SELECTOR → SUBJECT

Selector A	state = 'CA' AND city = 'SF'	→ orders.ca.sf.>
Selector B	state = 'CA' AND city = 'SF' AND price > 300	→ orders.ca.sf.*.high_value

WHY IT'S SIMPLER

Just different subscription strings	— no selector objects to create or manage
The subject says what will match	— no separate expression to cross-reference
Need payload rules?	— Synadia Protect can evaluate them at the gateway

```
name: us_trades
description: trades restrictions

facts:
  - connection_kind: client

conditions:
  - rule_type: message
  - subject_match: "trades.processed"

default: allow

rules:
  - success: deny
    fail: allow
    message: "trade spread exceeding 100"
    expression: |
      let trade = fromJSON(bytesToString(Message.Payload));
      debug(trade);
      trade.action == "sell" && (trade.high - trade.low) > 100
```

SYNADIA PROTECT

The gateway that protects NATS

Solace VPNs → NATS Accounts

WHY IT'S SIMPLER

- Decentralized accounts**
VPNs are broker-bound; NATS accounts span the supercluster
- One signed identity**
Solace stacks profiles before the namespace; NATS uses one user JWT
- Built-in gateways**
Cross-VPN traffic needs an external bridge; NATS gateways are built in
- Sharing at the JWT level**
A VPN bridge is a client connection; NATS uses imports and exports

SOLACE

- CP: Connections → Account connection limit
- CP: Subscriptions → User subscription limit
- CP: Guaranteed messaging → Account JetStream enabled
- CP: Endpoint create → Pub permission on JS API
- CP: Spool quota → JetStream storage limits
- CP: VPN bridging → Imports/exports, leaf nodes
- ACL: Topic pub/sub → User pub/sub permissions
- CP = client profile · ACL = ACL profile

NATS

MESSAGE VPN

- Client profile
Connection & rate limits
- ↓
- ACL profile
Topic pub/sub permissions
- ↓
- Topics & queues
The VPN's namespace

ACCOUNT

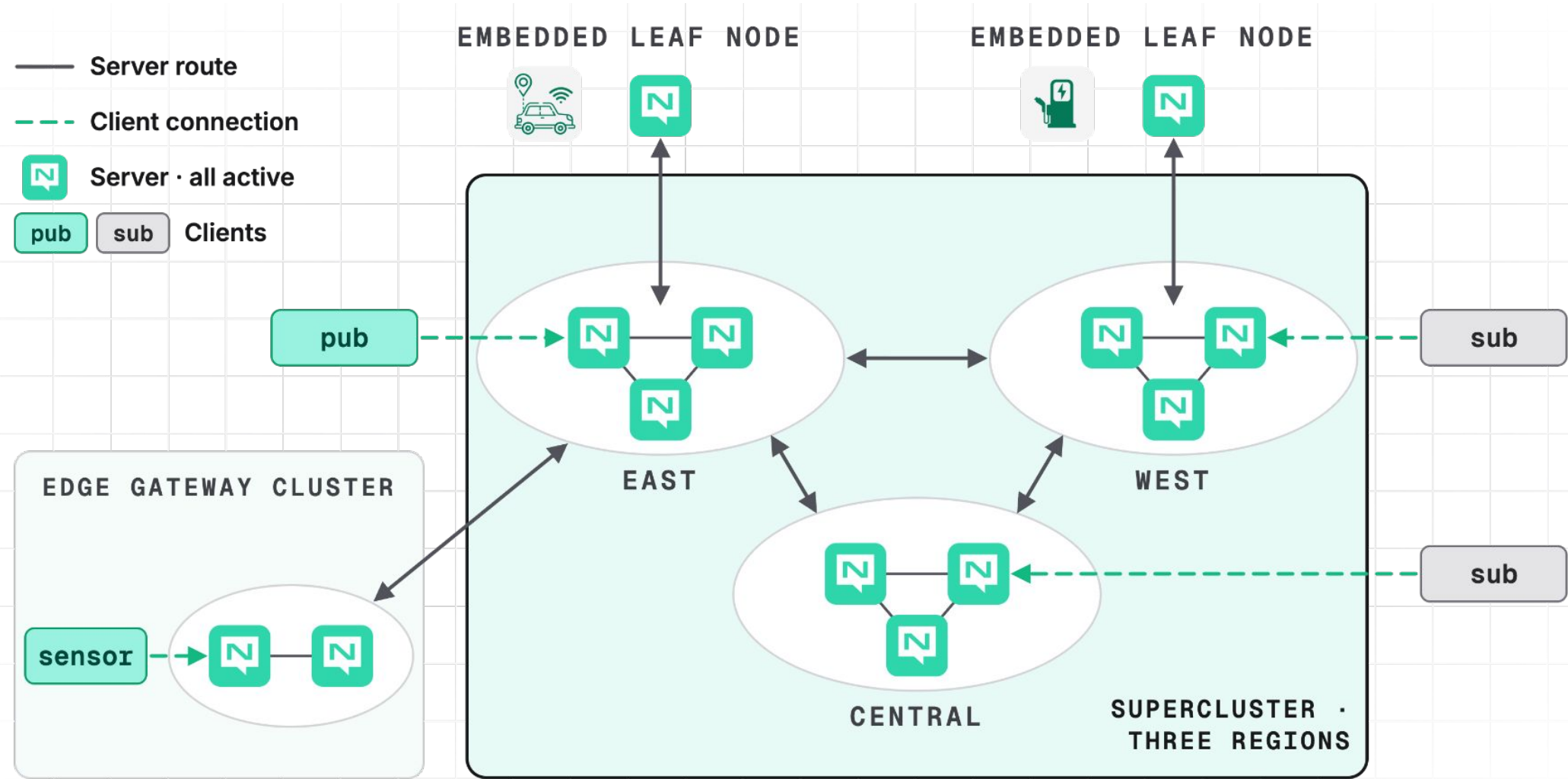
- User (JWT)
Connection + subject perms
- ↓
- Subjects
The account's namespace
- ↓
- Imports / exports
Native cross-account sharing

Partitioned Queues

→ Partitioned Consumer Groups

	SOLACE	NATS	SOLACE QUEUE	NATS GROUP
Storage	Physically divided into fixed partitions, one consumer each	Stream stays undivided; partitioning is a client-side library	Partitions Fixed count set at creation	Stream Undivided, single log
Partition model	Fixed at creation	Static (Kafka-like) or elastic, set by the consumer group	↓ Partition key Hashes each message to a partition	↓ Key pinning Pins each key to one worker
Hot standby	Built in	Built in	↓	↓
Delivery	Push	Pull		
Partition key	A message property	Part of the subject name	Bound consumer Preserves order per partition	Elastic workers Can join or leave anytime
Rebalancing	Automatic	Admin-driven		

Super Clusters & Leaf Nodes



What we're about to do

1. Selectors to Subjects
2. Solace VPNs to NATS accounts
3. Partitions groups
4. Leaf Node Topologies



Where we started, where we landed

WHY

Market demands require new messaging platforms that are scalable, can process large volumes of data, and can cover cloud, on-prem, and edge

HOW

We map the concepts, we realign the architecture, and adapt the applications/APIs

Next in the series

October 14th

Your MQTT Broker Is Holding You Back

Building edge-to-cloud systems with NATS

October 19th

Your Standby Cloud Won't Save You

Architecting with NATS JetStream

synadia.com/lp/enterprise-nats-use-cases

Questions

Bruno Baloi · bruno@synadia.com

synadia.com/lp/enterprise-nats-use-cases

Slides and resources sent within 48 hours.